

Job 1 :

Je commence par créer mon Repository sur mon Github. Puis, je vais créer un dossier "app", qui contiendra mon "index.html"

```
<> index.html X $ check_app.sh
C: > Users > lalal > Documents > gtacd > app > <> index.html > html
1  <!DOCTYPE html>
2  <html lang="fr">
3  <head>
4      <meta charset="UTF-8">
5      <title>GTACD - Mon Application</title>
6  </head>
7  <body>
8      <h1>Bienvenue sur mon application GTACD !</h1>
9      <p>Cette application est déployée automatiquement via un pipeline CI/CD sécurisé.</p>
10     <!-- Date de génération: sera ajoutée dynamiquement lors du déploiement -->
11 </body>
12 </html>
```

Ensuite, je vais faire un fichier qui s'appelle "check_app.sh"

```
<> index.html $ check_app.sh X
C: > Users > lalal > Documents > gtacd > app > $ check_app.sh
1  #!/bin/bash
2
3  echo "=== Début des vérifications de l'application ==="
4
5  if [ -f "app/index.html" ]; then
6      echo "OK : le fichier index.html est présent."
7      echo "=== Fin des vérifications : SUCCÈS ==="
8      exit 0
9  else
10     echo "ERREUR : le fichier index.html est introuvable."
11     echo "=== Fin des vérifications : ÉCHEC ==="
12     exit 1
13 fi
```

Je vais ensuite mettre tout ça sur Github, via le Terminal de VSCode.

```
✓ TERMINAL
PS C:\Users\lalel>
PS C:\Users\lalel> git add app/index.html app/check_app.sh
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\lalel> cd Documents\gtacd
PS C:\Users\lalel\Documents\gtacd> git add app/index.html app/check_app.sh
warning: in the working copy of 'app/check_app.sh', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\lalel\Documents\gtacd> git update-index --chmod=+x app/check_app.sh
PS C:\Users\lalel\Documents\gtacd> git commit -m "Job 1: ajout de l'application web minimale et du script de verification"
[main (root-commit) 3113629] Job 1: ajout de l'application web minimale et du script de verification
 2 files changed, 25 insertions(+)
 create mode 100755 app/check_app.sh
 create mode 100644 app/index.html
PS C:\Users\lalel\Documents\gtacd> git push origin main
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 786 bytes | 157.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/laleh-latte/gtacd.git
 * [new branch]      main -> main
PS C:\Users\lalel\Documents\gtacd>
```

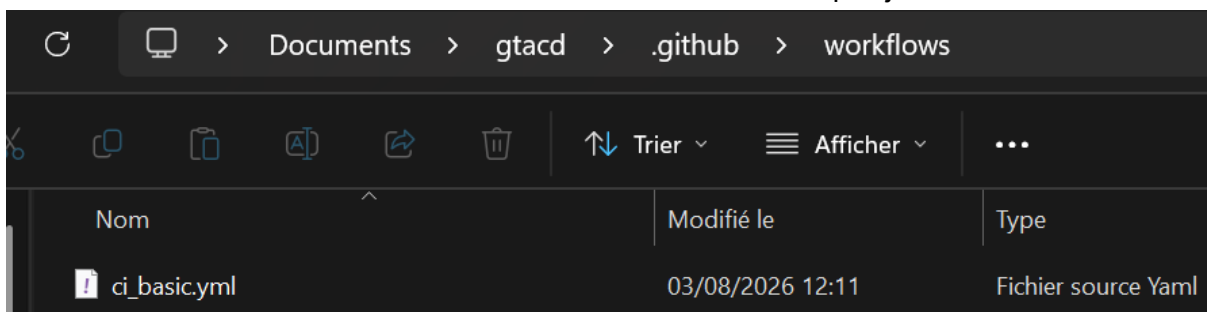
Comme on peut le voir, ça a bien été push. On va aller vérifier ça directement sur Github.

laleh-latte Job 1: ajout de l'application web minimale et du script de verification 3113629 · 4 minutes ago History		
Name	Last commit message	Last commit date
..		
check_app.sh	Job 1: ajout de l'application web minimale et du script de verif...	4 minutes ago
index.html	Job 1: ajout de l'application web minimale et du script de verif...	4 minutes ago

C'est tout bon !

Job 2 :

Je vais ensuite créer mon fichier YAML. Voici le chemin dans lequel je le mets :



En voici son contenu :

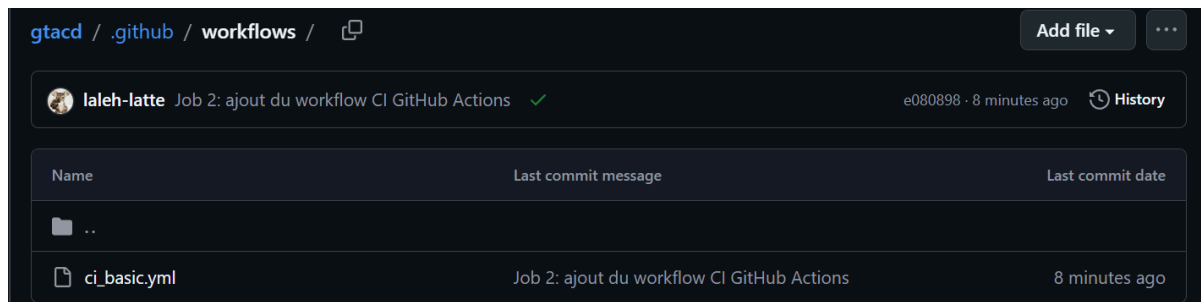
```
<> index.html    $ check_app.sh    ci_basic.yml X
C: > Users > lalal > Documents > gtacd > .github > workflows > ci_basic.yml
1  name: Basic CI Workflow
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    build-and-test:
10     runs-on: ubuntu-latest
11
12     steps:
13       - name: Checkout repository
14         uses: actions/checkout@v4
15
16       - name: Run application check script
17         run: ./app/check_app.sh
18
19       - name: Display index.html content
20         run: cat app/index.html
```

Ce workflow fait exactement ce qui est demandé : il se déclenche à chaque push sur main, un seul Job build-and-test sur un runner ubuntu-latest, et 3 étapes : checkout, exécution du script de vérif, affichage du contenu du HTML.

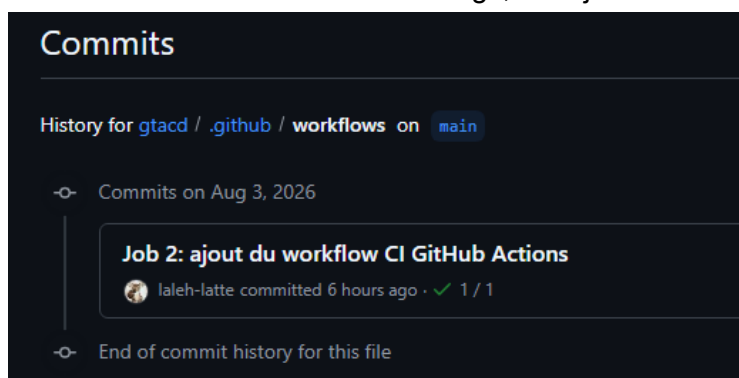
Maintenant, je vais envoyer tout ça sur Github.

```
PS C:\Users\lalal\Documents\gtacd> git add .github/workflows/ci_basic.yml
PS C:\Users\lalal\Documents\gtacd> git commit -m "Job 2: ajout du workflow CI Github Actions"
[main e080898] Job 2: ajout du workflow CI Github Actions
 1 file changed, 20 insertions(+)
 create mode 100644 .github/workflows/ci_basic.yml
PS C:\Users\lalal\Documents\gtacd> git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 591 bytes | 295.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/laleh-latte/gtacd.git
 3113629..e080898  main -> main
PS C:\Users\lalal\Documents\gtacd>
```

Et c'est bon, tout a été push aussi ! On va aller vérifier sur Github.



C'est tout bon ! Si on veut voir les logs, on a juste à voir l'historique. Le voici :



Job 3 :

Pourquoi utiliser les Secrets GitHub plutôt que d'écrire des informations sensibles en clair dans le code ?

Un dépôt GitHub, même privé, peut être vu par plusieurs personnes (collaborateurs, CI/CD, parfois des outils tiers connectés). Si un mot de passe, une clé API ou une clé SSH est écrit en clair dans un fichier :

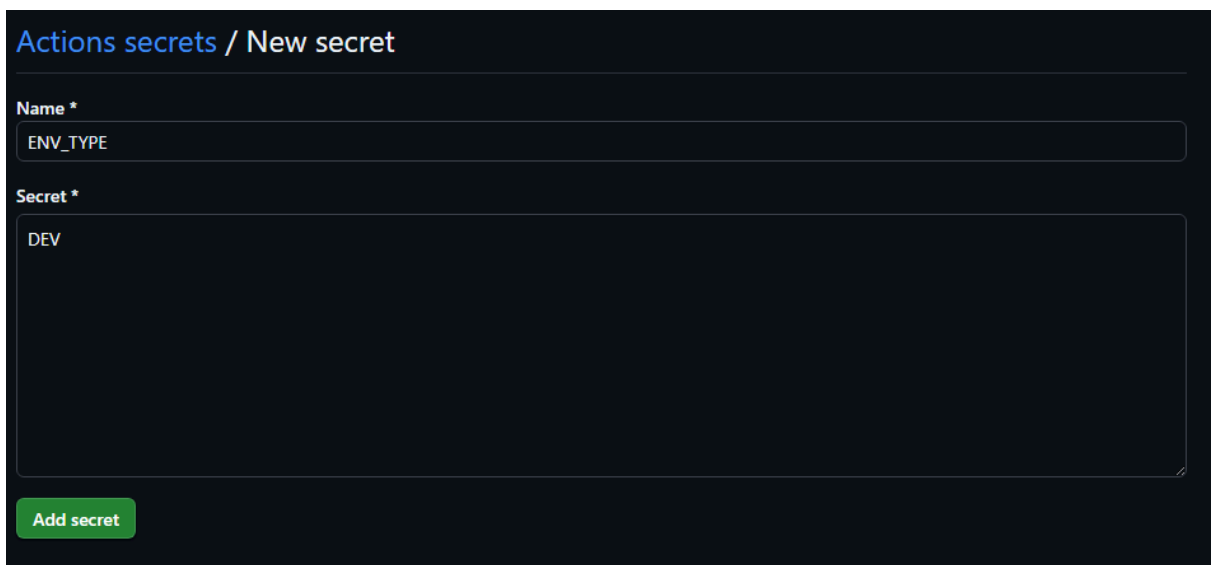
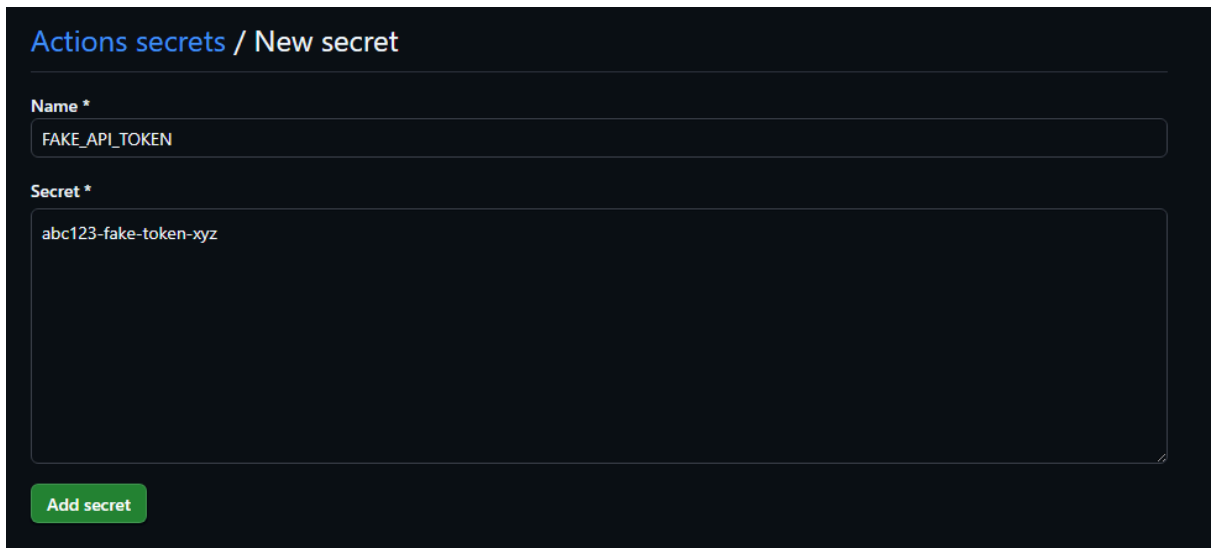
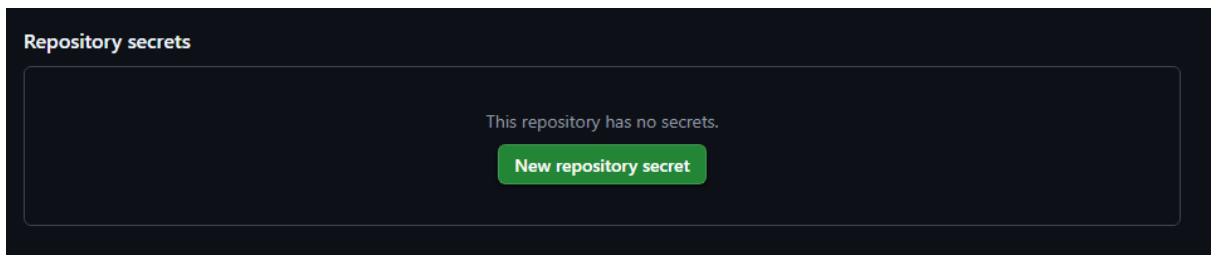
- Il reste **visible dans tout l'historique Git**, même si tu le supprimes plus tard (un `git log` ou une simple recherche dans les anciens commits suffit à le retrouver)
- Si le dépôt est **public** (comme le vôtre), n'importe qui sur Internet peut le voir immédiatement
- Des **bots scannent en permanence GitHub** à la recherche de clés API et tokens exposés, pour les exploiter (souvent en quelques minutes)

Les **GitHub Secrets**, eux, sont chiffrés, stockés séparément du code, et ne sont injectés que dans l'environnement d'exécution du workflow au moment voulu — jamais visibles dans le code source ni dans l'historique.

Quels sont les risques majeurs si un secret est exposé ?

- **Usurpation / accès non autorisé** : un attaquant qui récupère une clé API peut agir en votre nom (appels API illimités, facturation, actions destructrices)
- **Accès à l'infrastructure** : une clé SSH ou un mot de passe serveur exposé permet un accès direct à vos machines
- **Effet domino** : dans une chaîne CI/CD, un secret compromis (ex: token de déploiement) peut permettre de modifier le code déployé, empoisonnant toute la chaîne (c'est un vrai type d'attaque : *supply chain attack*)
- **Coûts financiers** : clé API cloud (AWS, GCP...) exposée = potentiellement des milliers d'euros de ressources utilisées frauduleusement
- Même après rotation du secret, **l'exposition initiale a eu lieu** — le mal est fait tant qu'on ne sait pas ce qu'un attaquant a pu en faire entre-temps

Je vais maintenant passer aux secrets GitHub. Je vais faire un nouveau Repository secret dans l'onglet "Secrets and variables".



Puis, dans mon fichier yaml, je vais rajouter ces lignes à la fin :

```

22     - name: Display secrets and environment variables
23       env:
24         FAKE_API_TOKEN: ${ secrets.FAKE_API_TOKEN }
25         ENV_TYPE: ${ secrets.ENV_TYPE }
26         CUSTOM_VAR: "Ceci est une variable personnalisée non-secrète"
27       run: |
28         echo "Variable personnalisée (non-secrète) : $CUSTOM_VAR"
29         echo "Variable prédéfinie GitHub (RUNNER_OS) : $RUNNER_OS"
30         echo "Valeur du secret FAKE_API_TOKEN : $FAKE_API_TOKEN"
31         echo "Valeur du secret ENV_TYPE : $ENV_TYPE"

```

Je peux ensuite push !

← Basic CI Workflow

✓ Job 3: ajout de l'affichage des secrets et variables d'environnement #2

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

Annotations

1 warning

build-and-test

succeeded 2 minutes ago in 7s

- Set up job
- Checkout repository
- Run application check script
- Display index.html content
- Display secrets and environment variables
 - Run echo "Variable personnalisée (non-secrète) : \$CUSTOM_VAR"
 - Variable personnalisée (non-secrète) : Ceci est une variable personnalisée non-secrète
 - Variable prédéfinie GitHub (RUNNER_OS) : Linux
 - Valeur du secret FAKE_API_TOKEN : ***
 - Valeur du secret ENV_TYPE : ***
- Post Checkout repository
- Complete job

On peut maintenant voir, dans mon onglet Actions, que dans mes secrets, il y a bien ce que je viens de coder ! Donc, tout fonctionne. On va pouvoir passer à la dernière partie du Job 3 : simuler un échec contrôlé.

```

app > $ check_app.sh
1  #!/bin/bash
2
3  echo "=== Début des vérifications de l'application ==="
4
5  if [ -f "app/index.html" ]; then
6      echo "OK : le fichier index.html est présent."
7      echo "=== Fin des vérifications : SUCCÈS ==="
8      exit 1
9  else
10     echo "ERREUR : le fichier index.html est introuvable."
11     echo "=== Fin des vérifications : ÉCHEC ==="
12     exit 1
13 fi

```

J'ai juste remplacé dans la ligne 8, l'ancien 0 du exit par un 1.

Triggered via push now
 laleh-latte pushed · 16937a6 · main
 Status: Failure
 Total duration: 7s
 Artifacts: -

ci_basicyml
 on: push

build-and-test 5s

Annotations
 1 error and 1 warning

build-and-test
 Process completed with exit code 1.

build-and-test
 Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: <https://github.blog/changelog/2023-09-19-deprecation-of-node-20-on-github-actions-runners/>

Et parfait, ça a forcé à faire échouer ! Je vais pouvoir remettre le script comme avant, pour la suite du projet.

All workflows
 Showing runs from all workflows

Filter workflow runs

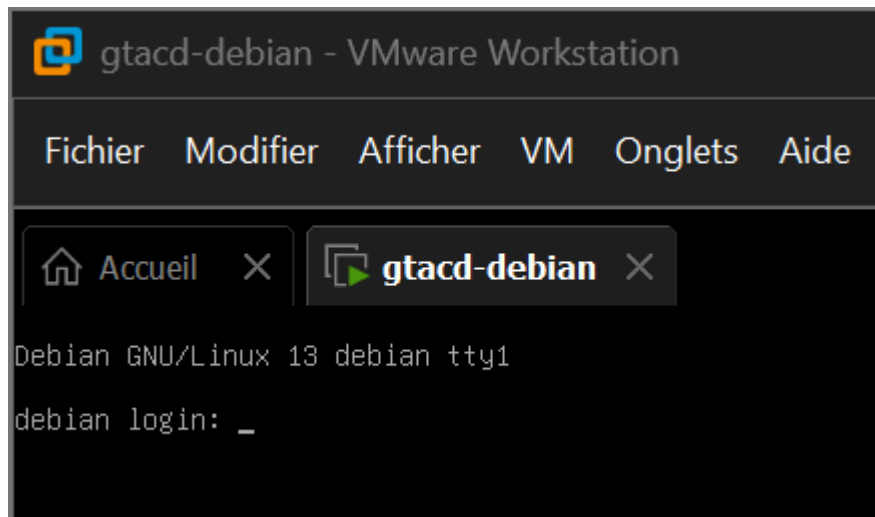
4 workflow runs

	Workflow	Event	Status	Branch	Actor
✓ Fix: retour du script de verification en etat de succes Basic CI Workflow #4: Commit b67ba48 pushed by laleh-latte	main		Success		
✗ Job 3: simulation d'un echec controle du script de verification Basic CI Workflow #3: Commit 16937a6 pushed by laleh-latte	main		Failure		
✓ Job 3: ajout de l'affichage des secrets et variables d'environnement Basic CI Workflow #2: Commit 6d5c3e3 pushed by laleh-latte	main		Success		
✓ Job 2: ajout du workflow CI GitHub Actions Basic CI Workflow #1: Commit e080898 pushed by laleh-latte	main		Success		

Parfait ! On voit que ça remarque correctement. Je vais pouvoir avancer.

Job 4 :

Je vais créer ma VM Debian pour les prochains Jobs.



Une fois faite, je peux continuer. On va enchaîner sur la configuration de l'utilisateur de déploiement.

```
root@debian:~# adduser deployuser
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
Modifier les informations associées à un utilisateur pour deployuser
Entrer la nouvelle valeur, ou appuyer sur ENTER pour la valeur par défaut
  NOM []:
  Numéro de chambre []:
  Téléphone professionnel []:
  Téléphone personnel []:
  Autre []:
Is the information correct? [Y/n] y
root@debian:~#
```

Puis, les commandes simples pour nous donner les outils.

```

tous les paquets sont à jour.
root@debian:~# apt install sudo -y
Installation de :
  sudo

Sommaire :
  Mise à niveau de : 0. Installation de : 1 Supprimé : 0. Non mis à jour : 0
Taille du téléchargement : 2 086 kB
Espace nécessaire : 6 866 kB / 17,5 GB disponible

Réception de : 1 http://deb.debian.org/debian trixie/main amd64 sudo amd64 1.9.16p2-3+deb13u2 [2 086 kB]
2 086 ko réceptionnés en 0s (9 205 ko/s)
Sélection du paquet sudo précédemment désélectionné.
(Lecture de la base de données... 35929 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../sudo_1.9.16p2-3+deb13u2_amd64.deb ...
Dépaquetage de sudo (1.9.16p2-3+deb13u2) ...
Paramétrage de sudo (1.9.16p2-3+deb13u2) ...
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.41-12+deb13u3) ...
root@debian:~# echo "deployuser ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/deployuser
root@debian:~# chmod 0440 /etc/sudoers.d/deployuser
root@debian:~# which python3
/usr/bin/python3
root@debian:~#

```

Et quand j'ai voulu me connecter en SSH, on voit bien que ça ne m'a rien demandé, et que je suis bien sudo.

```

PS C:\Users\lala\Documents\gtacd> ssh deployuser@192.168.88.132
deployuser@192.168.88.132's password:
Linux debian 6.12.100+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1
2.100-1 (2026-07-30) x86_64

The programs included with the Debian GNU/Linux system are free sof
tware;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
deployuser@debian:~$ sudo whoami
root
deployuser@debian:~$

```

Pourquoi est-il fondamental d'utiliser un utilisateur dédié avec le principe du moindre privilège plutôt que root directement ?

- Si le compte de déploiement automatisé est compromis (clé volée, faille dans le pipeline...), l'attaquant n'a accès qu'à ce que **deployuser** peut faire — pas un accès root total et illimité au système dès le départ.
- Ça limite la "surface de dégâts" : même avec sudo, on garde une trace claire de qui/quoi a fait quoi (**deployuser** dans les logs, plutôt que root anonyme).
- Root a accès à absolument tout le système (y compris des trucs qui n'ont rien à voir avec le déploiement) — un compte dédié réduit les risques d'erreur ou d'action destructrice non désirée.
- C'est une bonne pratique universelle en cyber : **principe du moindre privilège** = donner uniquement les droits nécessaires à la tâche, jamais plus.

Maintenant, on va passer à la suite du Job 4 : je vais générer la clé SSH dédiée.

```
PS C:\Users\lala> ssh-keygen -t rsa -b 2048 -f $env:USERPROFILE\.ssh\github_deploy_key -N ""
Generating public/private rsa key pair.
Your identification has been saved in C:\Users\lala\.ssh\github_deploy_key
Your public key has been saved in C:\Users\lala\.ssh\github_deploy_key.pub
The key fingerprint is:
SHA256:LJMcGk77aKrFLEwHUW6vIgtFznG1AaySwT6hmiVL8qY truc stupéfiant@pop-hip
The key's randomart image is:
+---[RSA 2048]-----+
| . . ++* . |
| .o . * B + |
| oo+.o O * |
| *Bo= + = . |
| +B= + o S |
| +.oo o . |
| .o. = . |
| E. = . |
| o |
+----[SHA256]-----+
PS C:\Users\lala>
```

Après avoir généré ma clé SSH, je vais pouvoir copier la clé publique vers ma VM.

```
PS C:\Users\lala> type $env:USERPROFILE\.ssh\github_deploy_key.pub
| ssh deployuser@192.168.88.132 "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
deployuser@192.168.88.132's password:
PS C:\Users\lala> ssh -i $env:USERPROFILE\.ssh\github_deploy_key deployuser@192.168.88.132
Linux debian 6.12.100+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.100-1 (2026-07-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Aug 4 08:50:43 2026 from 192.168.88.1
deployuser@debian:~$
```

Je suis bien connectée en SSH avec la clé, sans mot de passe et sans qu'on me redemande le mot de passe.

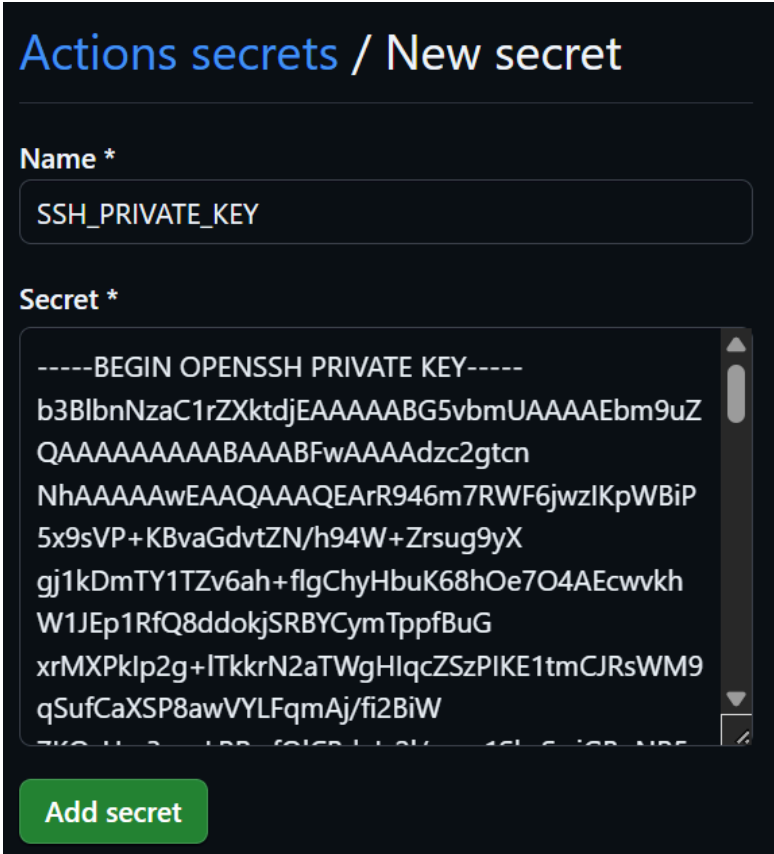
Pourquoi est-il crucial de ne pas utiliser ses clés SSH personnelles pour les déploiements automatisés ?

- Une clé personnelle donne souvent accès à plein d'autres systèmes (serveurs perso, autres projets, GitHub perso...) — si elle fuit via le pipeline CI/CD, c'est tout ça qui est compromis, pas juste ce projet
- Une clé dédiée = un **rayon d'action limité et connu** : elle ne sert qu'à une chose précise, donc en cas de fuite, l'impact est cantonné à ce projet

Quels sont les avantages d'avoir des clés dédiées par projet/environnement ?

- Facilite la **révocation ciblée** : si une clé est compromise, tu la supprimes juste de ce serveur-là, sans devoir changer tes clés persos partout
- Meilleure **traçabilité** : on sait exactement quelle clé sert à quoi
- Permet d'appliquer des restrictions spécifiques par clé si besoin (IP autorisée, commandes limitées, etc.)

Je vais maintenant me diriger sur Github pour créer 3 nouveaux secrets.



Actions secrets / New secret

Name *

SSH_PRIVATE_KEY

Secret *

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktbjEAAAAABG5vbmUAAAAEbm9uZ
QAAAAAAAAABAAAABFwAAAAAdzc2gtcn
NhAAAAAwEAAQAAQEA946m7RWF6jwzIKpWBiP
5x9sVP+KBvaGdvtZN/h94W+Zrsug9yX
gj1kDmTY1TZv6ah+flgChyHbuK68hOe7O4AEcwvk
W1JEp1RfQ8ddokjSRBYCymTppfBuG
xrMXPkIp2g+ITkkRN2aTWgHlqcZSzPIKE1tmCJR
WM9qSufCaXSP8awVYLFqmAj/fi2BiW
-----
```

Add secret

J'ai fait cette commande pour obtenir le résultat de ma clé : type
\$env:USERPROFILE\.ssh\github_deploy_key

Name *

SERVER_IP

Secret *

192.168.88.132

Name *

SERVER_USER

Secret *

deployuser

Maintenant que les 3 secrets sont en place, je vais pouvoir passer à la dernière partie du Job 4 : créer le workflow de simulation.

```
secure-deployment-simulation.yml U X
.github > workflows > secure-deployment-simulation.yml
1  name: Secure Deployment Simulation
2
3  on:
4    push:
5      branches:
6        - main
7    pull_request:
8      branches:
9        - main
10
11 jobs:
12   validate-deployment-config:
13     runs-on: ubuntu-latest
14
15     steps:
16       - name: Checkout repository
17         uses: actions/checkout@v4
18
19       - name: Validate SSH private key format and strength
20         env:
21           SSH_PRIVATE_KEY: ${ secrets.SSH_PRIVATE_KEY }
22         run: |
23           echo "=== Validation de la clé SSH privée ==="
24           echo "$SSH_PRIVATE_KEY" > /tmp/deploy_key
25           chmod 600 /tmp/deploy_key
26
27           if ssh-keygen -l -f /tmp/deploy_key > /dev/null 2>&1; then
28             echo "OK : la clé est dans un format valide."
29           else
```

```

30     echo "ERREUR : format de clé invalide."
31     exit 1
32 fi
33
34 KEY_BITS=$(ssh-keygen -l -f /tmp/deploy_key | awk '{print $1}')
35 echo "Taille de la clé : $KEY_BITS bits"
36 if [ "$KEY_BITS" -lt 2048 ]; then
37     echo "ERREUR : la clé fait moins de 2048 bits."
38     exit 1
39 else
40     echo "OK : la taille de la clé respecte le minimum requis (>= 2048 bits)."
41 fi
42
43 if ssh-keygen -y -f /tmp/deploy_key -P "" > /dev/null 2>&1; then
44     echo "OK : la clé n'est pas protégée par une passphrase."
45 else
46     echo "ERREUR : la clé semble protégée par une passphrase (incompatible avec l'automatisation)."
47     exit 1
48 fi
49
50 rm -f /tmp/deploy_key
51
52 - name: Validate server configuration
53   env:
54     SERVER_IP: ${ secrets.SERVER_IP }
55     SERVER_USER: ${ secrets.SERVER_USER }
56   run: |

```

```

57     echo "=== Validation de la configuration serveur ==="
58     if [ -z "$SERVER_IP" ]; then
59         echo "ERREUR : SERVER_IP n'est pas défini."
60         exit 1
61     else
62         echo "OK : SERVER_IP est défini."
63     fi
64
65     if [ -z "$SERVER_USER" ]; then
66         echo "ERREUR : SERVER_USER n'est pas défini."
67         exit 1
68     else
69         echo "OK : SERVER_USER est défini (utilisateur : $SERVER_USER)."
70     fi
71
72     if [[ "$SERVER_IP" =~ ^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$ ]]; then
73         echo "OK : le format de SERVER_IP ressemble à une adresse IPv4 valide."
74     else
75         echo "ATTENTION : SERVER_IP ne ressemble pas à une IPv4 standard."
76     fi
77
78 - name: Simulate SSH connection and deployment commands
79   env:
80     SERVER_IP: ${ secrets.SERVER_IP }
81     SERVER_USER: ${ secrets.SERVER_USER }
82   run: |
83     echo "=== Simulation de connexion SSH (aucune connexion réelle) ==="
84     echo "Commande qui serait exécutée : ssh $SERVER_USER@$SERVER_IP"
85     echo ""

```

```

86     echo "=== Commandes de déploiement qui seraient lancées ==="
87     echo "1. Installation de Nginx sur le serveur"
88     echo "2. Création du répertoire /var/www/html/tp-app"
89     echo "3. Copie des fichiers de app/ vers le serveur"
90     echo "4. Application du template Nginx (nginx_vhost.conf.j2)"
91     echo "5. Redémarrage du service Nginx"
92
93     - name: Security best practices check
94       run: |
95         echo "=== Contrôles de sécurité ==="
96         echo "OK : utilisation d'un utilisateur dédié non-root pour le déploiement."
97         echo "OK : clé SSH dédiée, différente des clés personnelles."
98         echo "OK : secrets stockés via GitHub Secrets, jamais en clair dans le code."
99         echo "OK : clé privée sans passphrase, mais accès restreint via authorized_keys sur le serveur cible"
100        echo "Validation de la robustesse de la configuration : terminée avec succès."

```

Je vais maintenant pouvoir ajouter ce fichier à Git, dans mon prochain commit.

```

PS C:\Users\lalal\Documents\gtacd> git add .github/workflows/secure-
deployment-simulation.yml
PS C:\Users\lalal\Documents\gtacd> git commit -m "Job 4: ajout du w
orkflow de simulation de deploiem
t securise
[main 41bd27b] Job 4: ajout du workflow de simulation de deploiem
t securise
 1 file changed, 100 insertions(+)
 create mode 100644 .github/workflows/secure-deployment-simulation.
yml
PS C:\Users\lalal\Documents\gtacd> git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.67 KiB | 1.67 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/laleh-latte/gtacd.git
 b67ba48..41bd27b  main -> main
PS C:\Users\lalal\Documents\gtacd>

```

C'est bon ! Tout est push. Comme on peut le voir, ça correspond bien aux 100 lignes que j'ai créées.



Et c'est bon ! On voit bien que ça fonctionne correctement, et que ça a bien été déployé. Il me reste une dernière étape : le script de test local que je vais exécuter sur ma machine.

```
secure-deployment-simulation.yml  test_deployment_local.ps1 U X
test_deployment_local.ps1
1  $SERVER_IP = "192.168.88.132"
2  $SERVER_USER = "deployuser"
3  $SSH_KEY = "$env:USERPROFILE\.ssh\github_deploy_key"
4
5  Write-Host "=== Test de connexion SSH avec la nouvelle cle ===" -ForegroundColor Cyan
6  ssh -i $SSH_KEY -o StrictHostKeyChecking=no "$SERVER_USER@$SERVER_IP" "echo 'Connexion SSH reussie.'"
7
8  Write-Host "`n=== Test des droits sudo ===" -ForegroundColor Cyan
9  ssh -i $SSH_KEY "$SERVER_USER@$SERVER_IP" "sudo whoami"
10
11 Write-Host "`n=== Test de creation d'un repertoire d'application ===" -ForegroundColor Cyan
12 ssh -i $SSH_KEY "$SERVER_USER@$SERVER_IP" "sudo mkdir -p /var/www/html/tp-app-test && echo 'Repertoire cree av"
13
14 Write-Host "`n=== Verification des services systeme disponibles ===" -ForegroundColor Cyan
15 ssh -i $SSH_KEY "$SERVER_USER@$SERVER_IP" "systemctl list-units --type=service --state=running | head -n 10"
16
17 Write-Host "`n=== Tests termines ===" -ForegroundColor Green
```

Je crée ce fichier à la racine de mon dossier gtacd.


```

PS C:\Users\lalal\Documents\gtacd> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
PS C:\Users\lalal\Documents\gtacd> .\test_deployment_local.ps1
=== Test de connexion SSH avec la nouvelle cle ===
Connexion SSH reussie.

=== Test des droits sudo ===
root

=== Test de creation d'un repertoire d'application ===
Repertoire cree avec succes.

=== Verification des services systeme disponibles ===
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
cron.service                        loaded active running Regular background program processing daemon
dbus.service                        loaded active running D-Bus System Message Bus
getty@tty1.service                  loaded active running Getty on tty1
open-vm-tools.service               loaded active running Service for virtual machines hosted on VMware
ssh.service                         loaded active running OpenBSD Secure Shell server
systemd-journald.service             loaded active running Journal Service
systemd-logind.service               loaded active running User Login Management
systemd-timesyncd.service            loaded active running Network Time Synchronization
systemd-udevd.service                loaded active running Rule-based Manager for Device Events and Files

=== Tests termines ===

```

Et comme on peut le voir, après avoir outrepassé le fait que Powershell nous bloque, tous les tests passent ; connexion SSH, sudo, création de répertoire et services système.

Job 5 :

Je vais commencer par installer Ansible sur mon pc.

```

bl00kle@pop-hip:~$ ansible --version
ansible [core 2.20.1]
  config file = None
  configured module search path = ['/home/bl00kle/.ansible/plugins
/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/
ansible
  ansible collection location = /home/bl00kle/.ansible/collections
:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.14.4 (main, Apr  8 2026, 04:02:31) [GCC 15.2.
0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)

```

C'est bon ! On peut voir ma version d'Ansible (la 2.20.1).
Je fais un dossier "ansible" à la racine de mon gtacd, et j'y code ceci :

```

ansible >  ansiblenginx_vhost.conf.j2
1  server {
2      listen 80;
3      server_name {{ ansible_host }};
4
5      root /var/www/html/tp-app;
6      index index.html;
7
8      location / {
9          try_files $uri $uri/ =404;
10     }
11 }

```

J'utilise `ansible_host`, une variable Ansible automatique qui contient l'IP du serveur cible.
Puis, je fais un deuxième fichier.

ansible > ! deploy.yml

```
1  ---
2  - name: Déploiement de l'application web
3    hosts: all
4    become: true
5
6    tasks:
7      - name: Installer Nginx
8        apt:
9          name: nginx
10         state: present
11         update_cache: true
12
13      - name: Créer le répertoire de destination de l'application
14        file:
15          path: /var/www/html/tp-app
16          state: directory
17          owner: www-data
18          group: www-data
19          mode: '0755'
20
21      - name: Copier les fichiers de l'application
22        copy:
23          src: ../app/
24          dest: /var/www/html/tp-app/
25          owner: www-data
26          group: www-data
27          mode: '0644'
28
29      - name: Configurer Nginx via le template Jinja2
```

```

29 - name: Configurer Nginx via le template Jinja2
30   template:
31     src: nginx_vhost.conf.j2
32     dest: /etc/nginx/sites-available/tp-app
33   notify: Redémarrer Nginx
34
35 - name: Activer le site (lien symbolique)
36   file:
37     src: /etc/nginx/sites-available/tp-app
38     dest: /etc/nginx/sites-enabled/tp-app
39     state: link
40   notify: Redémarrer Nginx
41
42 - name: Désactiver le site par défaut de Nginx
43   file:
44     path: /etc/nginx/sites-enabled/default
45     state: absent
46   notify: Redémarrer Nginx
47
48 handlers:
49   - name: Redémarrer Nginx
50     service:
51       name: nginx
52       state: restarted

```

Maintenant, je vais créer mon fichier inventory.ini à la racine de mon dossier gtacd.

```

inventory.ini
1  [servers]
2  192.168.88.132 ansible_user=deployuser ansible_ssh_private_key_file=/mnt/c/Users/lalal/.ssh/github_deploy_key

```

Je retourne ensuite dans mon WSL pour ping, et :

```
bl00kle@pop-hip:/mnt/c/Users/lalal/Documents/gtacd$ ansible all -i
inventory.ini -m ping
[WARNING]: Host '192.168.88.132' is using the discovered Python in
terpreter at '/usr/bin/python3.13', but future installation of ano
ther Python interpreter could cause a different interpreter to be
discovered. See https://docs.ansible.com/ansible-core/2.20/referen
ce_appendices/interpreter_discovery.html for more information.
192.168.88.132 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.13"
  },
  "changed": false,
  "ping": "pong"
}
```

Ansible communique bien avec ma VM ! Je peux maintenant lancer le playbook de déploiement.

Avec la commande `ansible-playbook -i inventory.ini ansible/deploy.yml`, je lance le playbook.

```
PLAY RECAP *****
*****
192.168.88.132      : ok=8    changed=4    unreachable=0
failed=0    skipped=0    rescued=0    ignored=0
```

failed=0 et ok=8, signifient que mon playbook s'est exécuté avec succès. Mon app est maintenant déployée sur la VM via Ansible, avec Nginx configuré et le site activé. Je vais vérifier que ça marche vraiment, grâce au navigateur de mon ordinateur.



La classe...

```
PS C:\Users\laleh\Documents\gtacd> git commit -m "Job 5: creation d
u playbook Ansible et du template Nginx"
[main 6ca6395] Job 5: creation du playbook Ansible et du template N
ginx
 3 files changed, 64 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 ansible/deploy.yml
 create mode 100644 ansible/nginx_vhost.conf.j2
PS C:\Users\laleh\Documents\gtacd> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.09 KiB | 223.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/laleh-latte/gtacd.git
 9e4c6bd..6ca6395  main -> main
```

Je finis par push tous mes fichiers (les trois fichiers ajoutés : .gitignore, deploy.yml et nginx_vhost.conf.j2).

Job 6 :

Je commence par renommer mon fichier "ci_basis.yml" par "ci_cd_full.yml". Je remplace ensuite l'entièreté de son contenu par :

```
.github > workflows > ci_cd_full.yml
```

```
1  name: CI/CD Full Pipeline
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    build-and-test:
10     runs-on: ubuntu-latest
11
12     steps:
13       - name: Checkout repository
14         uses: actions/checkout@v4
15
16       - name: Run application check script
17         run: ./app/check_app.sh
18
19       - name: Display index.html content
20         run: cat app/index.html
21
22       - name: Display secrets and environment variables
23         env:
24           FAKE_API_TOKEN: ${ secrets.FAKE_API_TOKEN }
25           ENV_TYPE: ${ secrets.ENV_TYPE }
26           CUSTOM_VAR: "Ceci est une variable personnalisée non-secrète"
27         run: |
28           echo "Variable personnalisée (non-secrète) : $CUSTOM_VAR"
29           echo "Variable prédéfinie GitHub (RUNNER_OS) : $RUNNER_OS"
```

```
30     echo "Valeur du secret FAKE_API_TOKEN : $FAKE_API_TOKEN"
31     echo "Valeur du secret ENV_TYPE : $ENV_TYPE"
32
33   deploy:
34     needs: build-and-test
35     runs-on: ubuntu-latest
36
37     steps:
38       - name: Checkout repository
39         uses: actions/checkout@v4
40
41       - name: Set up SSH agent
42         uses: webfactory/ssh-agent@v0.9.0
43         with:
44           ssh-private-key: ${ secrets.SSH_PRIVATE_KEY }
45
46       - name: Create Ansible inventory file
47         env:
48           SERVER_IP: ${ secrets.SERVER_IP }
49           SERVER_USER: ${ secrets.SERVER_USER }
50         run: |
51           echo "[servers]" > inventory.ini
52           echo "$SERVER_IP ansible_user=$SERVER_USER ansible_ssh_common_args='-o StrictHostKeyChecking=no'"
```

```

53 |         chmod 600 inventory.ini
54 |
55 |     - name: Install Ansible
56 |       run: pip install ansible
57 |
58 |     - name: Run Ansible playbook
59 |       env:
60 |         SERVER_IP: ${ secrets.SERVER_IP }
61 |         SERVER_USER: ${ secrets.SERVER_USER }
62 |       run: ansible-playbook -i inventory.ini ansible/deploy.yml

```

Ce que j'ai fait : needs: build-and-test -> le job deploy ne se lance QUE si le job CI a réussi. webfactory/ssh-agent charge ma clé privée dans un agent SSH sur le runner, sans jamais l'écrire en clair dans un fichier visible. L'inventaire est généré dynamiquement à chaque run, avec chmod 600 pour restreindre les permissions.

The screenshot shows a GitHub Actions workflow run for a job named 'deploy'. The status is 'failed now in 43s'. A search bar for logs is present. The 'Run Ansible playbook' step is expanded, showing a failure at 11s. The log output includes:

```

1 | ▶ Run ansible-playbook -i inventory.ini ansible/deploy.yml
9 |
10 | PLAY [Déploiement de l'application web] *****
11 |
12 | TASK [Gathering Facts] *****
13 | [ERROR]: Task failed: Failed to connect to the host via ssh: ssh: connect to host *** port 22: Connection timed out
14 | fatal: [***]: UNREACHABLE! => {"changed": false, "msg": "Task failed: Failed to connect to the host via ssh: ssh:
15 | connect to host *** port 22: Connection timed out", "unreachable": true}
16 |
17 | PLAY RECAP *****
18 | *** : ok=0 changed=0 unreachable=1 failed=0 skipped=0 rescued=0 ignored=0
19 | Error: Process completed with exit code 4.

```

Quand je push, ça fait exactement ce que j'anticipais : Connection timed out sur le port 22. Les serveurs Github Actions tournent sur Internet et ne peuvent pas atteindre l'IP locale de ma VM, qui n'est visible que depuis mon propre réseau. Au Job 4, il est écrit "En raison des contraintes réseau (portail captif), nous utiliserons une approche de simulation...", donc ce comportement est normal et attendu. Pas une erreur de ma part.

Je vais documenter l'échec :

- Le job `build-and-test` (CI) a fonctionné parfaitement.
- Le job `deploy` a échoué avec un timeout de connexion SSH (Connection timed out sur le port 22).

- **Cause** : l'IP de la VM (192.168.88.132) est une adresse locale, uniquement accessible depuis le réseau local — les runners GitHub Actions, hébergés sur internet, ne peuvent pas l'atteindre.
- **Ce que ça prouve pourtant** : toute la chaîne de configuration est correcte (secrets bien configurés, agent SSH bien mis en place, playbook fonctionnel comme prouvé au Job 5 en exécution manuelle) — seul l'aspect réseau bloque, ce qui est cohérent avec les contraintes annoncées dès le Job 4.
- **Solutions réelles en production** : héberger sur un serveur avec IP publique (cloud), utiliser un runner **self-hosted** (une machine que tu contrôles, sur le même réseau que la VM, qui exécute les jobs GitHub Actions), ou un tunnel VPN/sécurisé entre les deux réseaux.

Job 7 :

Je commence par changer le contenu de mon app/index.html pour indiquer une nouvelle version.

```
app > < index.html > p
1 <p>Cette application est déployée automatiquement via un pipeline CI/CD sécurisé. V2.0 Déployé !</p>
```

Puis, je vais commit et push.

Triggered via push 1 minute ago

 **laleh-latte** pushed  `de5dd0c` **main**

Status	Total duration	Artifacts
Failure	53s	—

ci_cd_full.yml
on: push

 **build-and-test**
5s

 **deploy**
40s

[-] [] [+]

C'est encore une fois exactement ce qu'on attendait : le build-and-test est vert, la CI fonctionne donc parfaitement, mais c'est le deploy qui échoue pour la raison réseau qu'on a identifiée et documentée.

Le schéma montre bien aussi la dépendance entre les deux jobs (needs: build-and-test) : deploy ne s'est lancé qu'après le succès de "build-and-test", exactement comme configuré.

Job 8 :

Récapitulatif des étapes clés et outils utilisés

J'ai construit un pipeline CI/CD complet en 2 parties :

- **CI (GitHub Actions)** : build-and-test avec checkout, script de vérification, gestion de secrets.
- **CD (Ansible + GitHub Actions)** : déploiement automatisé avec utilisateur dédié, clé SSH dédiée, playbook Ansible, template Jinja2.

Outils utilisés : GitHub Actions, Ansible, Nginx, SSH, WSL/PowerShell.

Automatisation complète : à chaque push sur `main`, la CI se déclenche automatiquement, teste l'app, et tente le déploiement — sans intervention manuelle (hormis la contrainte réseau qu'on a documentée).

Pour aller plus loin en sécurité CI/CD (questions de réflexion) :

Scans SAST/SCA — Intégrer des outils comme *CodeQL* (SAST, analyse le code source pour détecter des failles) ou *Dependabot/Snyk* (SCA, scanne les dépendances tierces pour des vulnérabilités connues) directement comme étapes du job `build-and-test`, avant même le déploiement.

Tests DAST — Après un déploiement réussi, lancer un outil comme *OWASP ZAP* contre l'app en ligne, pour détecter des failles exploitables en conditions réelles (injections, XSS...).

Audits et logs — Les logs GitHub Actions et Ansible tracent qui a déclenché quoi, quand, et le résultat exact de chaque étape — précieux pour reconstituer une chronologie en cas d'incident, ou repérer un déploiement suspect (heure inhabituelle, échecs répétés...).

Gestion des environnements — En prod réelle : séparer dev/staging/prod avec des workflows différents, exiger une **approbation manuelle** avant tout déploiement en prod (feature *GitHub Environments*), isoler les credentials de prod des autres environnements.

Stratégies de Rollback — Toujours garder la possibilité de revenir rapidement à la version précédente (versionner les déploiements, garder l'ancienne config Nginx en backup) — indispensable si un déploiement casse la prod.